

Part 2: Evidence of successful completion of the activities and learning

Exercise 1.2.1

What types of networks do you use for your daily activities?

My Arch Linux desktop (wired), my home server / previous computer (wired), phone (wireless), and other devices communicate through my router. They are connected over either Ethernet or WiFi.

My home network configuration is quite interesting. My GL.iNet SFT1200 operates in wireless repeater / WISP mode, using the landlord's WiFi as its WAN uplink to create a separate private LAN for my home server and other devices. It performs its own NAT, DHCP, firewalling, routing, and local DNS resolution on its own subnet, thus isolating my network from my landlord's LAN. This basically creates a nested network.

How many different types of network connections do you use on a daily basis?

I normally use four main types every day. They are Ethernet, WiFi, 4G/5G cellular, and Bluetooth BR/EDR and sometimes Bluetooth BLE.

Exercise 1.2.2

Can you use layered architecture to explain any other complicated system that you use on daily basis?

A web application would be a really good example. At the top, I interact with the user interface through my web browser. The browser will then communicate with the application layer, which will handle the website logic and user requests. The backend will then process the request, communicate with databases or any other external APIs/services as necessary, and then will pass the response back through the same layers. Underneath this, the operating system, network protocols and the physical connection handles the actual data transfer. Each layer has a separate responsibility, making the overall system more modular.

Exercise 1.6.1

What else was captured? For this you can capture your own traffic.

As seen below, my computer performed a standard DNS query for `cdn.jsdelivr.net` and got those responses.

The image shows a Wireshark packet capture window titled 'sample.pcapng'. The packet list pane shows four packets. Packet 392 is a DNS standard query for 'cdn.jsdelivr.net'. Packet 393 is a DNS standard query response for 'cdn.jsdelivr.net'. Packet 400 is a DNS standard query response for 'cdn.jsdelivr.net'. Packet 402 is a DNS standard query response for 'cdn.jsdelivr.net'. The packet details pane shows the structure of the DNS query and response. The query is for 'cdn.jsdelivr.net: type A, class IN'. The response contains three records: 'cdn.jsdelivr.net: type CNAME, class IN, cname cdn.jsdelivr.net.cdn.cloudflare.net', 'cdn.jsdelivr.net.cdn.cloudflare.net: type A, class IN, addr 104.17.207.5', and 'cdn.jsdelivr.net.cdn.cloudflare.net: type A, class IN, addr 104.17.208.5'. The packet bytes pane shows the raw data of the DNS query and response.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
392	11.792101008		192.168.8.136	192.168.8.1	DNS	76	Standard query 0xb2db A cdn.jsdelivr.net
393	11.792106889	0.000005881	192.168.8.136	192.168.8.1	DNS	76	Standard query response 0xd7d8 AAAA cdn.jsdelivr.net
400	11.804132098	0.012025209	192.168.8.1	192.168.8.136	DNS	222	Standard query response 0xd7d8 AAAA cdn.jsdelivr.net CNAME jsdelivr.map.fastl...
402	11.805379686	0.001247588	192.168.8.1	192.168.8.136	DNS	154	Standard query response 0xb2db A cdn.jsdelivr.net CNAME cdn.jsdelivr.net.cdn...

Queries

- > cdn.jsdelivr.net: type A, class IN

Answers

- > cdn.jsdelivr.net: type CNAME, class IN, cname cdn.jsdelivr.net.cdn.cloudflare.net
- > cdn.jsdelivr.net.cdn.cloudflare.net: type A, class IN, addr 104.17.207.5
- > cdn.jsdelivr.net.cdn.cloudflare.net: type A, class IN, addr 104.17.208.5

[Request In: 392]
[Time: 13.278678 milliseconds]

Exercise 1.6.2

Try using both the filter syntax as shown above, and the expression builder, to find filters for the following conditions: All frames containing the DNS protocol;

The best way to filter out DNS traffic is to use the `dns` filter.

The screenshot shows the Wireshark interface with the filter 'dns' applied. The packet list displays several DNS frames. The selected packet (No. 105) is a standard query for 'api2.cursor.sh'. The packet details pane shows the Ethernet II, Internet Protocol Version 4, and User Datagram Protocol (UDP) layers. The UDP layer shows Source Port: 39115 and Destination Port: 53. The packet bytes pane shows the raw data of the packet.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
90	5.369721338		192.168.8.136	192.168.8.1	DNS	74	Standard query 0x6f3f A www.google.com
91	5.369736045	0.000014707	192.168.8.136	192.168.8.1	DNS	74	Standard query 0xf324 AAAA www.google.com
95	5.370963993	0.001227948	192.168.8.1	192.168.8.136	DNS	202	Standard query response 0x6f3f A www.google.com A 142.251.155.119 A 142.
96	5.371355877	0.000391884	192.168.8.1	192.168.8.136	DNS	298	Standard query response 0xf324 AAAA www.google.com AAAA 2001:4860:48c7
104	5.429377930	0.058022053	192.168.8.136	192.168.8.1	DNS	74	Standard query 0xd0d2 A api2.cursor.sh
105	5.429388509	0.000010579	192.168.8.136	192.168.8.1	DNS	74	Standard query 0xcfcc AAAA api2.cursor.sh
106	5.430799741	0.001411232	192.168.8.1	192.168.8.136	DNS	267	Standard query response 0xd0d2 A api2.cursor.sh CNAME api2geo.cursor.sh
107	5.431158753	0.000359012	192.168.8.1	192.168.8.136	DNS	139	Standard query response 0xcfcc AAAA api2.cursor.sh CNAME api2geo.cursor.sh

As seen above, all DNS traffic goes through UDP port 53. Therefore, `udp.port == 53` filter should also work. But note that it might display any other UDP traffic that's sent over port 53 as well.

The screenshot shows the Wireshark interface with the filter 'udp.port == 53' applied. The packet list displays a larger set of DNS frames, including queries and responses for various domains. The selected packet (No. 106) is a standard query response for 'api2.cursor.sh'. The packet details pane shows the Ethernet II, Internet Protocol Version 4, and User Datagram Protocol (UDP) layers. The UDP layer shows Source Port: 53 and Destination Port: 39115. The packet bytes pane shows the raw data of the packet.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
90	5.369721338		192.168.8.136	192.168.8.1	DNS	74	Standard query 0x6f3f A www.google.com
91	5.369736045	0.000014707	192.168.8.136	192.168.8.1	DNS	74	Standard query 0xf324 AAAA www.google.com
95	5.370963993	0.001227948	192.168.8.1	192.168.8.136	DNS	202	Standard query response 0x6f3f A www.google.com A 142.251.155.119 A 142.
96	5.371355877	0.000391884	192.168.8.1	192.168.8.136	DNS	298	Standard query response 0xf324 AAAA www.google.com AAAA 2001:4860:48c7
104	5.429377930	0.058022053	192.168.8.136	192.168.8.1	DNS	74	Standard query 0xd0d2 A api2.cursor.sh
105	5.429388509	0.000010579	192.168.8.136	192.168.8.1	DNS	74	Standard query 0xcfcc AAAA api2.cursor.sh
106	5.430799741	0.001411232	192.168.8.1	192.168.8.136	DNS	267	Standard query response 0xd0d2 A api2.cursor.sh CNAME api2geo.cursor.sh
107	5.431158753	0.000359012	192.168.8.1	192.168.8.136	DNS	139	Standard query response 0xcfcc AAAA api2.cursor.sh CNAME api2geo.cursor.sh
121	5.543771266	0.112612513	192.168.8.136	192.168.8.1	DNS	75	Standard query 0xaad8 A ssl.gstatic.com
122	5.556255311	0.012484045	192.168.8.1	192.168.8.136	DNS	91	Standard query response 0xaad8 A ssl.gstatic.com A 172.217.211.94
123	5.556301377	0.000046066	192.168.8.136	192.168.8.1	DNS	75	Standard query 0xa9da AAAA ssl.gstatic.com
124	5.568416832	0.012115455	192.168.8.1	192.168.8.136	DNS	103	Standard query response 0xa9da AAAA ssl.gstatic.com AAAA 2404:6800:400a:
205	6.164948740	0.596531908	192.168.8.136	192.168.8.1	DNS	71	Standard query 0x7776 A hirusha.xyz
206	6.164960062	0.000011322	192.168.8.136	192.168.8.1	DNS	71	Standard query 0x2474 AAAA hirusha.xyz
207	6.166348069	0.001388007	192.168.8.1	192.168.8.136	DNS	135	Standard query response 0x7776 A hirusha.xyz A 185.199.108.153 A 185.199

All frames sent by your local machine;

As seen here, my local IP address for this interface connected to my host machine is 192.168.8.136.

```
[athena@pc ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp9s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether b0:82:e2:0b:8d:70 brd ff:ff:ff:ff:ff:ff
    altname enx082e20b8d70
    inet 192.168.8.136/24 brd 192.168.8.255 scope global dynamic noprefixroute enp9s0
        valid_lft 25719sec preferred_lft 25719sec
    inet6 fe80::7e3c:742a:b145:8329/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: wlp8s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether b2:4d:21:9f:d1:39 brd ff:ff:ff:ff:ff:ff permaddr 50:fe:0c:6a:b4:46
    altname wlx50fe0c6ab446
[athena@pc ~]$
```

To filter by the source IP address, you can use this filter: `ip.src == 192.168.8.136`

The image shows a Wireshark packet capture interface. The filter bar at the top contains the filter `ip.src == 192.168.8.136`. The packet list table below shows the following data:

No.	Time	Delta	Source	Destination	Protocol	Length	Info
2	0.025484856		192.168.8.136	142.251.119.95	UDP	75	44634 → 443 Len=33
4	0.215985879	0.190501023	192.168.8.136	162.159.134.234	TCP	66	56756 → 443 [ACK] Seq=1 Ack=59 Win=644 Len=0 TSval=2458287204 TSecr=2709
6	0.216189791	0.000203912	192.168.8.136	162.159.134.234	TCP	66	56756 → 443 [ACK] Seq=1 Ack=105 Win=644 Len=0 TSval=2458287205 TSecr=270
8	0.217104623	0.000914832	192.168.8.136	162.159.134.234	TCP	66	56756 → 443 [ACK] Seq=1 Ack=151 Win=644 Len=0 TSval=2458287206 TSecr=270
10	0.217704576	0.000599953	192.168.8.136	162.159.134.234	TCP	66	56756 → 443 [ACK] Seq=1 Ack=197 Win=644 Len=0 TSval=2458287206 TSecr=270
12	0.242255153	0.024550577	192.168.8.136	162.159.134.234	TCP	66	56756 → 443 [ACK] Seq=1 Ack=243 Win=644 Len=0 TSval=2458287231 TSecr=270
13	1.054167823	0.811912670	192.168.8.136	104.16.76.142	TLSv1...	105	Application Data
14	1.054213238	0.000045415	192.168.8.136	104.16.137.209	TLSv1...	105	Application Data
15	1.054451814	0.000238576	192.168.8.136	104.16.137.209	TLSv1...	90	Application Data
16	1.054461202	0.000009388	192.168.8.136	104.16.137.209	TCP	66	40382 → 443 [FIN, ACK] Seq=64 Ack=1 Win=74 Len=0 TSval=1965062178 TSecr=
17	1.054670924	0.000209722	192.168.8.136	104.16.76.142	TLSv1...	90	Application Data
18	1.054677356	0.000006432	192.168.8.136	104.16.76.142	TCP	66	39026 → 443 [FIN, ACK] Seq=64 Ack=1 Win=72 Len=0 TSval=1082342465 TSecr=
21	1.065352185	0.010674829	192.168.8.136	104.16.137.209	TCP	66	40382 → 443 [ACK] Seq=65 Ack=2 Win=74 Len=0 TSval=1965062189 TSecr=29952
24	1.066083904	0.000731719	192.168.8.136	104.16.76.142	TCP	66	39026 → 443 [ACK] Seq=65 Ack=2 Win=72 Len=0 TSval=1082342477 TSecr=35202
25	1.303105745	0.237021841	192.168.8.136	104.18.18.125	TLSv1...	248	Application Data

The packet details pane at the bottom shows the selected packet (No. 25) with the following details:

- Destination Port (udp.dstport), 2 bytes: 0020 77 5f ae 5a 01 bb 00 29 cf c5 4a ed f9 b1 9c 74 w..Z...t
- Packets: 2574 · Displayed: 1162 (45.1%) · Dropped: 0 (0.0%) · Profile: Default

—> Next Page

Then, I visited <http://neverssl.com/> , which is a public website on the internet that can be accessed without HTTPS. This lets us see the traffic clearly.

The image displays a Wireshark packet capture of an HTTP GET request and its response. The main packet list shows several HTTP requests, with packet 2440 being the selected GET request to neverssl.com. The packet details pane for packet 2440 shows the Ethernet II, Internet Protocol Version 4, and Hypertext Transfer Protocol layers. The packet bytes pane shows the raw data of the packet. The packet details pane for the selected packet (2440) shows the HTTP GET request details, including the Host, User-Agent, Accept, Accept-Language, Accept-Encoding, Referer, Connection, Upgrade-Insecure-Requests, and Priority. The packet details pane for the selected packet (2440) also shows the HTTP response details, including the status code (200 OK), Date, Server, Upgrade, Connection, Last-Modified, ETag, Vary, Accept-Ranges, Content-Encoding, Content-Length, Keep-Alive, and Content-Type. The packet details pane for the selected packet (2440) also shows the HTML response body, which includes a title, a style, and a body with a font-family, font-size, color, and margin.

Wireshark - Follow HTTP Stream (tcp.stream eq 62) - enp9s0

GET / HTTP/1.1
Host: neverssl.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:152.0) Gecko/20100101 Firefox/152.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Referer: https://www.google.com/
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, l

HTTP/1.1 200 OK
Date: Tue, 14 Jul 2026 00:53:30 GMT
Server: Apache/2.4.66 ()
Upgrade: h2,h3c
Connection: Upgrade, Keep-Alive
Last-Modified: Wed, 29 Jun 2022 00:23:33 GMT
ETag: "f79-5e28b29d38e93-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 1900
Keep-Alive: timeout=5, max=100
Content-Type: text/html; charset=UTF-8

<html>
 <head>
 <title>NeverSSL - Connecting ... </title>
 <style>
 body {
 font-family: Montserrat, helvetica, arial, sans-serif;
 font-size: 16px;
 color: #444444;
 margin: 0;
 }
 h2 {
 font-size: 24px;
 }

As seen above, you can see the response received for the HTTP GET request sent.